# PureBasic OOP Reference Manual (English)

Welcome to the official documentation of the Object-Oriented Programming (OOP) transpiler for PureBasic.

---

## 1. Introduction

PureBasic OOP brings modern Object-Oriented syntax while retaining the high execution speed, small footprint, and native compil

### Key Features:

- **Classes & Methods**: Clean object declaration with `Class` and `Method`.
- **Single Inheritance**: Class inheritance using the `Extends` keyword.
- **Dynamic VTable Polymorphism**: Automatic resolution of virtual method tables and method overriding.
- **Parent Invocation `Super::`**: Reuse parent class method implementations with `Super::Method(...)`.
- **Encapsulation**: Access control for fields and methods (`Public`, `Protected`, `Private`).
- **Constructors & Destructors**: Instantiation via `New ClassName(...)` and memory release via `*obj\Free()`.

---

## 2. Syntax & Grammar

### 2.1 Class Declaration and Inheritance

```oop
; Base class
Class Animal
    Protected nom.s
    Protected age.i

    Public Method Init(nom_p.s, age_p.i)
    Public Method Crier()
    Public Method Free()
EndClass

; Derived class
Class Chien Extends Animal
    Protected race.s

    Public Method Init(nom_p.s, age_p.i, race_p.s)
    Public Method Crier()              ; Overrides parent method
    Public Method Rapporter(objet.s)   ; Specific new method
    Public Method Free()               ; Destructor
EndClass
```

### 2.2 Method Implementation and `Super::`

Methods access their internal fields using the `This` instance pointer.
To invoke a parent implementation, use `Super::`.

```oop
Method Animal::Init(nom_p.s, age_p.i)
    This\nom = nom_p
    This\age = age_p
EndMethod

Method Animal::Crier()
    PrintN("[Animal] " + This\nom + " makes a generic sound.")
EndMethod
```

```
Method Animal::Free()
    FreeStructure(This)
EndMethod

Method Chien::Init(nom_p.s, age_p.i, race_p.s)
    Super::Init(nom_p, age_p) ; Initialize parent fields
    This\race = race_p
EndMethod

Method Chien::Crier()
    Super::Crier() ; Call parent behavior
    PrintN("   ==> DOG (" + This\race + ") : Woof ! Woof !")
EndMethod

Method Chien::Rapporter(objet.s)
    PrintN(This\nom + " fetches: " + objet)
EndMethod

Method Chien::Free()
    Super::Free()
EndMethod
```

### 2.3 Polymorphic Usage

```oop
Define *monChien.Chien = New Chien("Buddy", 4, "Retriever")
*monChien\Crier()

; Dynamic Polymorphism via parent-type list / pointer
NewList *refuge.Animal()
AddElement(*refuge()) : *refuge() = *monChien

ForEach *refuge()
    *refuge()\Crier() ; Dynamically dispatches to Chien::Crier()
    *refuge()\Free()  ; Clean polymorphic destructor dispatch
Next
```

## 3. Generated PureBasic Plumbing

The transpiler maps OOP abstractions to native PureBasic constructs:
1. `Interface Chien_vt Extends Animal_vt` (declaring only added methods).
2. `Structure Chien_Inst Extends Animal_Inst` (with `*VTable` pointer header).
3. `DataSection` containing the function pointers with overridden methods in proper slots.
4. Factory constructors `New_Chien(...)` initializing structure instances and VTable references.

## 4. Build & Execution Guide

### Transpile `.pbo` to `.pb`:

```cmd
"compiler/transpiler.exe" "src/my_file.pbo" "src/my_file_generated.pb"
```

### Compile the generated PureBasic code:

```cmd
```

```
"C:\Program Files\PureBasic\Compilers\pbcompiler.exe" "src/my_file_generated.pb" /CONSOLE /EXE "src/my_file.exe"
```